

Lecture – 4

Efficient Binary Trees- Part 2

M.G.Noel A.S Fernando (PhD)

Professor

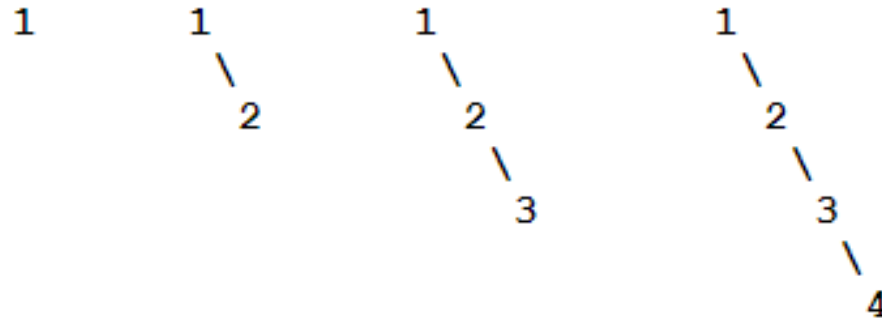
NSBM/Dept. of Information Systems Engineering, UCSC



Balanced Binary Search Trees

Balanced Binary Search Trees cont..

- Binary search trees allow binary search for fast lookup, insertion and deletion of data items, and can be used to implement dynamic sets and lookup tables
- Binary search trees fail to achieve our goal of performing lookup, insertion, and deletion in $O(\log_2(n))$ time when there are n items in the tree
- Imagine starting with an empty tree and inserting 1, 2, 3 and 4, in that order.



Balanced Binary Search Trees cont..

- You do not get a branching tree, but a linear tree.
- All of the left subtrees are empty. Because of this behavior, *in the worst case* each of the operations (lookup, insertion and deletion) takes time $\Theta(n)$.
- From the perspective of the worst case, we might as well be using a linked list and linear search.
- That bad worst-case behavior can be avoided by using an idea called *height balancing*, sometimes called *AVL trees*

What are Self-Balancing Binary Search Trees?

- A self-balancing BST maintains minimal height after each operation to optimize performance. Though rebalancing may seem costly, it ensures faster future operations, making it efficient overall.
- A Binary tree with Height h can have at most $2^0 + 2^1 + \dots + 2^h = 2^{(h+1)} - 1$
- $n \leq 2^{(h+1)} - 1$
- $h \geq \lceil \log_2(n+1) \rceil - 1 \geq \lfloor \log_2(n) \rfloor$

Hence, for self-balancing BSTs, the minimum height must always be $\log_2(n)$ rounded down. Moreover, a binary tree is said to be **balanced** if the height of left and right children of every node differ by either **-1**, **0** or **+1**. This value is known as the **balance factor**.

Balance factor = Height of the left subtree - Height of the right subtree

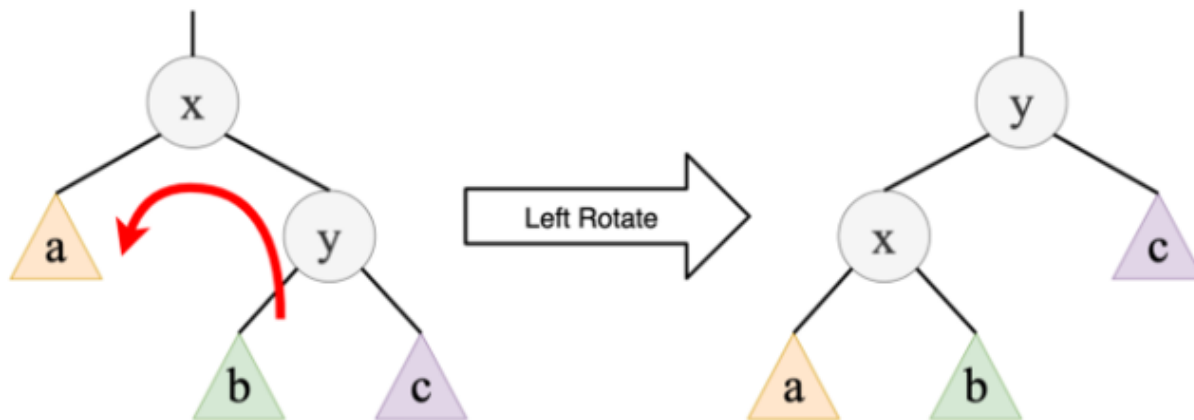
How do Self-Balancing Binary Search Trees Balance?

Self-balancing BSTs use rotations after insertions and deletions to maintain balance without breaking the BST property.

- There are two types of rotation operations used to balance BSTs without violating the binary search tree property.

1. Left rotation

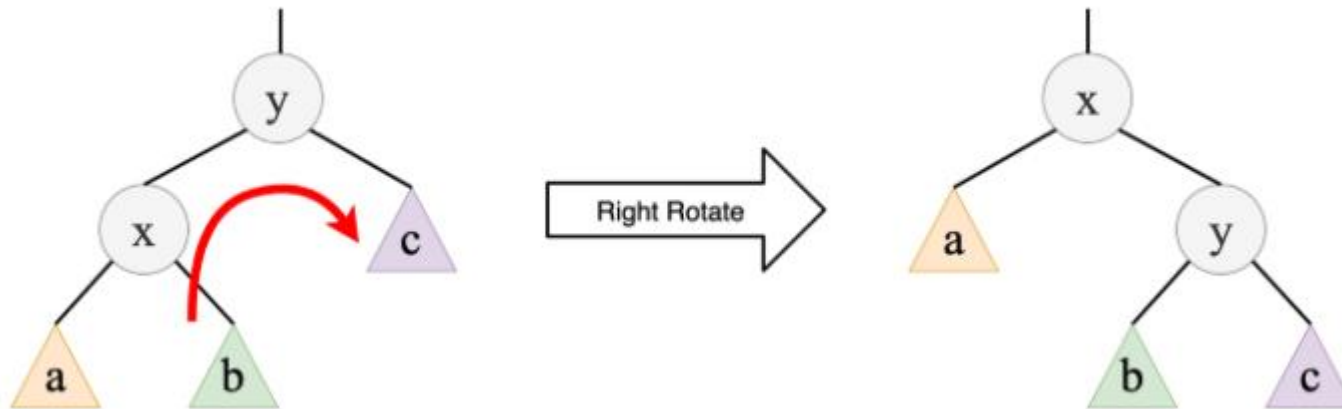
- When we left rotate about node x , node y becomes the new root of the subtree. Node x becomes the left child of node y and subtree b becomes the right child of node x



How do Self-Balancing Binary Search Trees Balance? Cont..

2. Right rotation

When we right rotate about node y , node x becomes the new root of the subtree. Node y becomes the right child of node x and subtree b becomes the left child of node y .



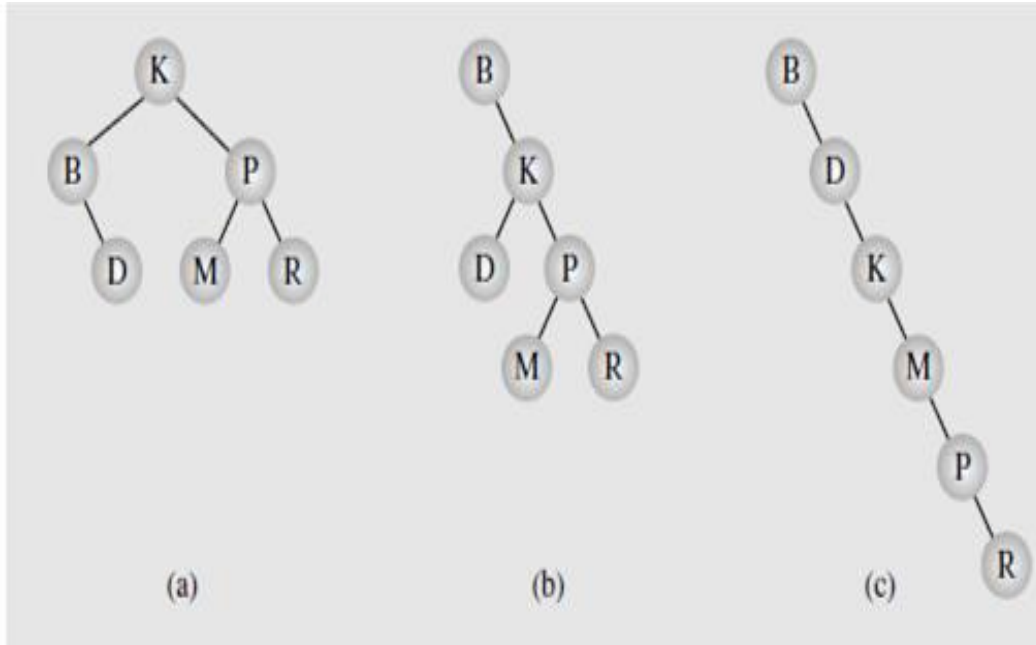
Types of Self-Balancing Binary Search Trees

Below are a few types of self-balancing BSTs.

1. AVL trees
2. Red-black trees
3. Splay trees
4. Treaps (A Randomized Binary Search Tree)

More detail about the tree balancing

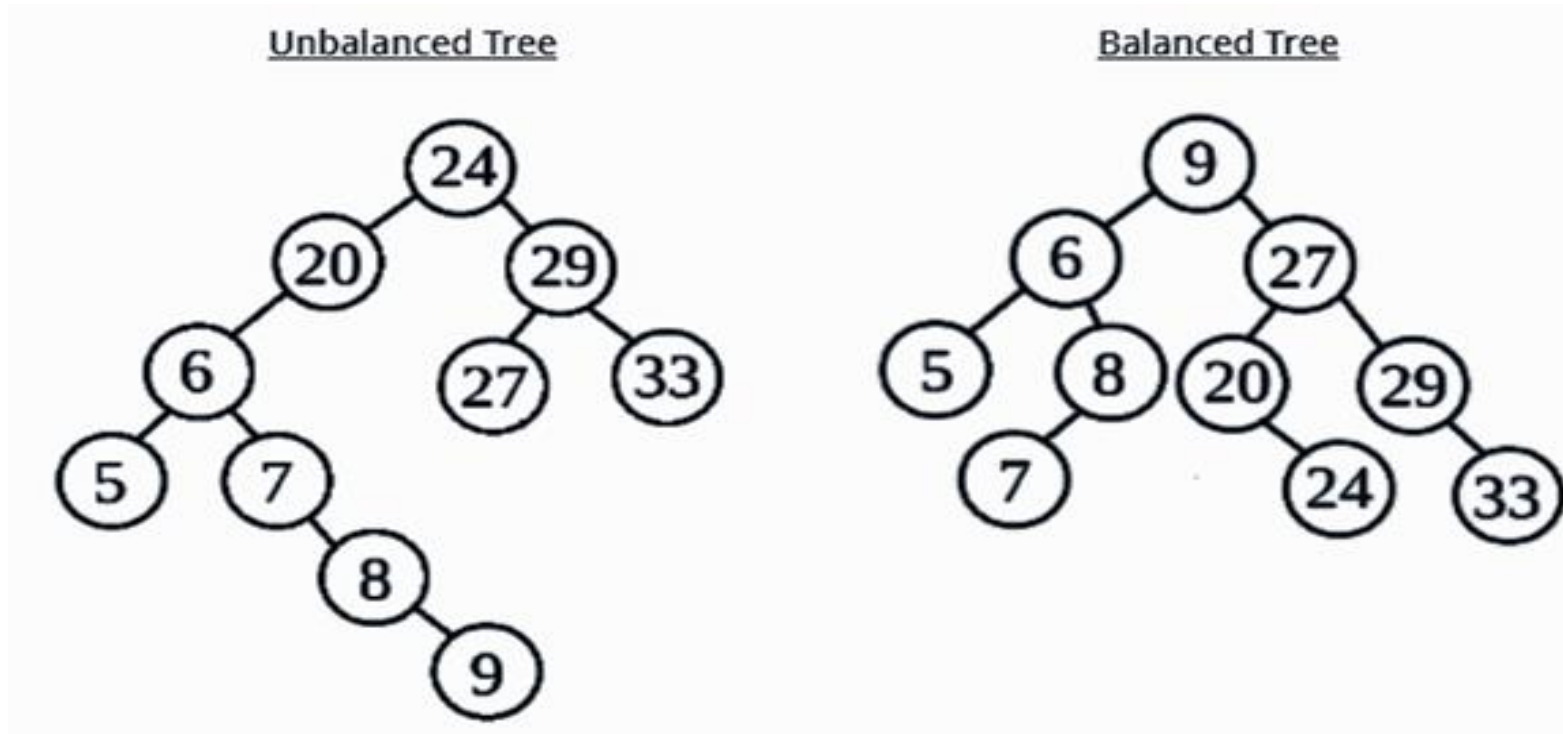
- Consider the following three examples.



- A binary tree is balanced if the height difference between the subtrees of any node is at most one; larger differences indicate an unbalanced tree.
- For example, Node K in Figure (b) is balanced, while Node B in Figure (b) is unbalanced.
- For the same node B in (c), the difference is the worst possible, namely, five. Additionally, a tree is considered perfectly balanced if it is balanced and all leaves are located on either one level or two adjacent levels.

More detail about the tree balancing

- Examples



Global tree balancing

- **Balancing a tree**
- We have seen the searching benefits of a balanced tree, but how do we balance a tree?
- Consider a tree with 10,000 nodes
- At its least efficient (linked list) at most 10,000 elements need to be tested find the element
- When balance, just 14 tests need to be performed
- The height of the tree is $\lg(10,000)=13.2877123795=\text{approx.. } 14$

Creating a balanced tree

- Assume all the data is in a stored array.
 - ✓ The middle element becomes the root
 - ✓ The middle element of one half becomes a child
 - ✓ The middle element of the other half becomes the other child.
 - ✓ Then recursive.....
 - ✓ E.g. 1,2,3,4,5,6,7 (elements stored in an array)

How to balance?

```
Template < T>
```

```
Void BST <T>:: Balance (T Data[],  
int First, int last){
```

```
  If (first<=last) {
```

```
    int middle=(first+last)/2;
```

```
    insert(data[middle]);
```

```
    balance(data, first,middle-1);
```

```
    balance (data,middle+1, last);
```

```
  }
```

```
}
```

- Weakness of this method
- This algorithm is quite inefficient as it relies on using extra space to store an array of values, and all values must be in this array(perhaps by in-order traversal).
- Let look at some alternative.

Global tree balancing -The DSW (Day-Stout-Warren) algorithm

- The DSW algorithm, created by Colin Day and improved by Stout and Warren, balances a binary tree by first converting it into a linked-list-like structure.
- The main operation used for tree transformation in the algorithm is tree rotation, which involves rotating a child node around its parent.
- There are two symmetrical types of rotations in the algorithm: left and right rotations.

Global tree balancing -The DSW algorithm

- 1. First create a backbone or vine.
- 2. Tree balancing :The steps involved in this compound operation are shown in the image below. (Then you transform the backbone into nicely balance tree.)

Right rotation of child ch about parent Par.



The right rotation of the node Ch about its parent Par is performed according to the following tree:

Algorithms

■ createBackbone(root, n)

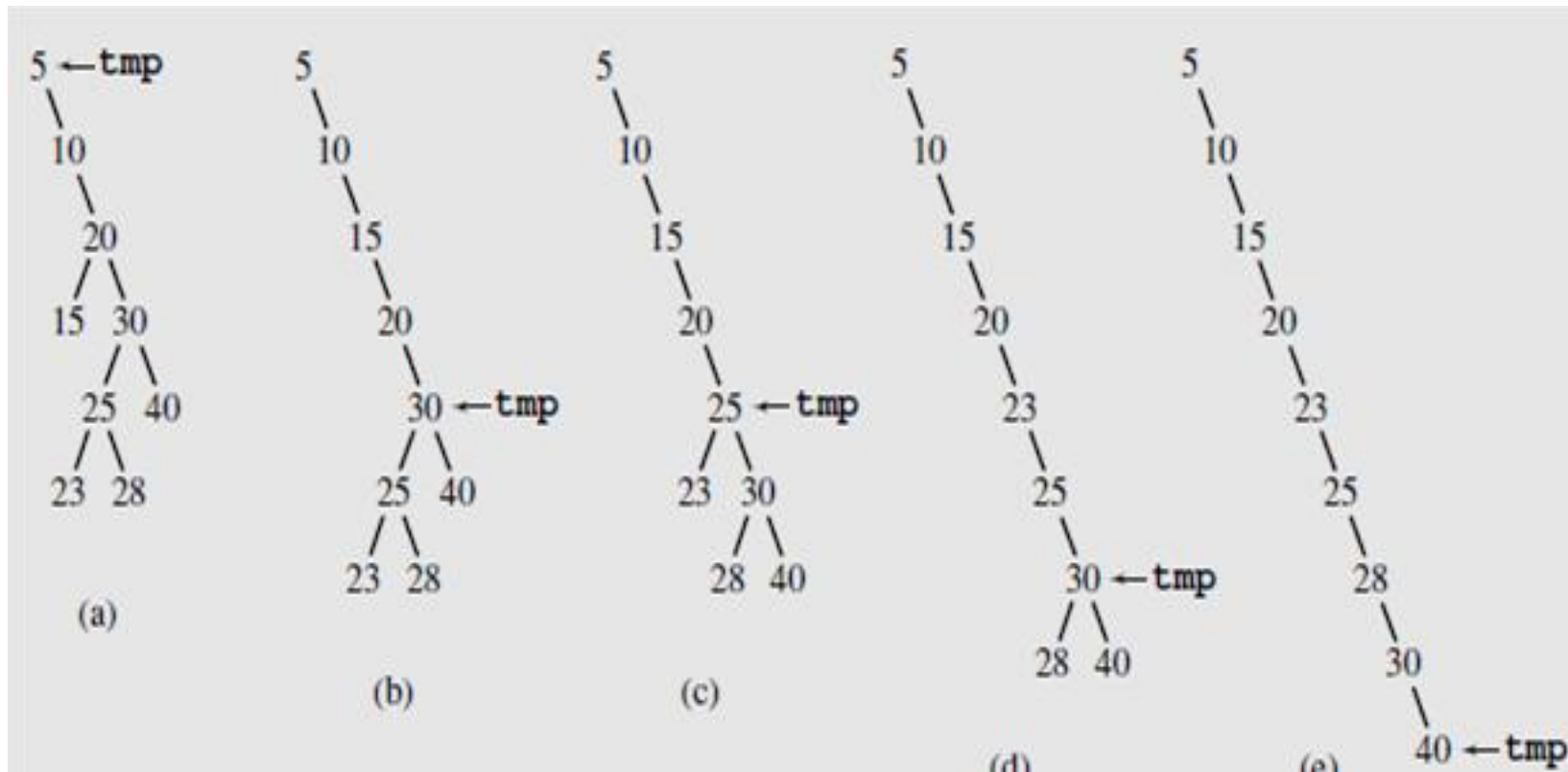
- Tmp = root
- While (Tmp != 0)
 - If Tmp has a left child
 - Rotate this child about Tmp
 - Set Tmp to the child which just became parent
 - Else set Tmp to its right child

■ createPerfectTree(n)

- $M = 2^{\text{floor}[\lg(n+1)]} - 1$;
- Make n-M rotations starting from the top of the backbone;
- While (M > 1)
 - $M = M/2$;
 - Make M rotations starting from the top of the backbone;

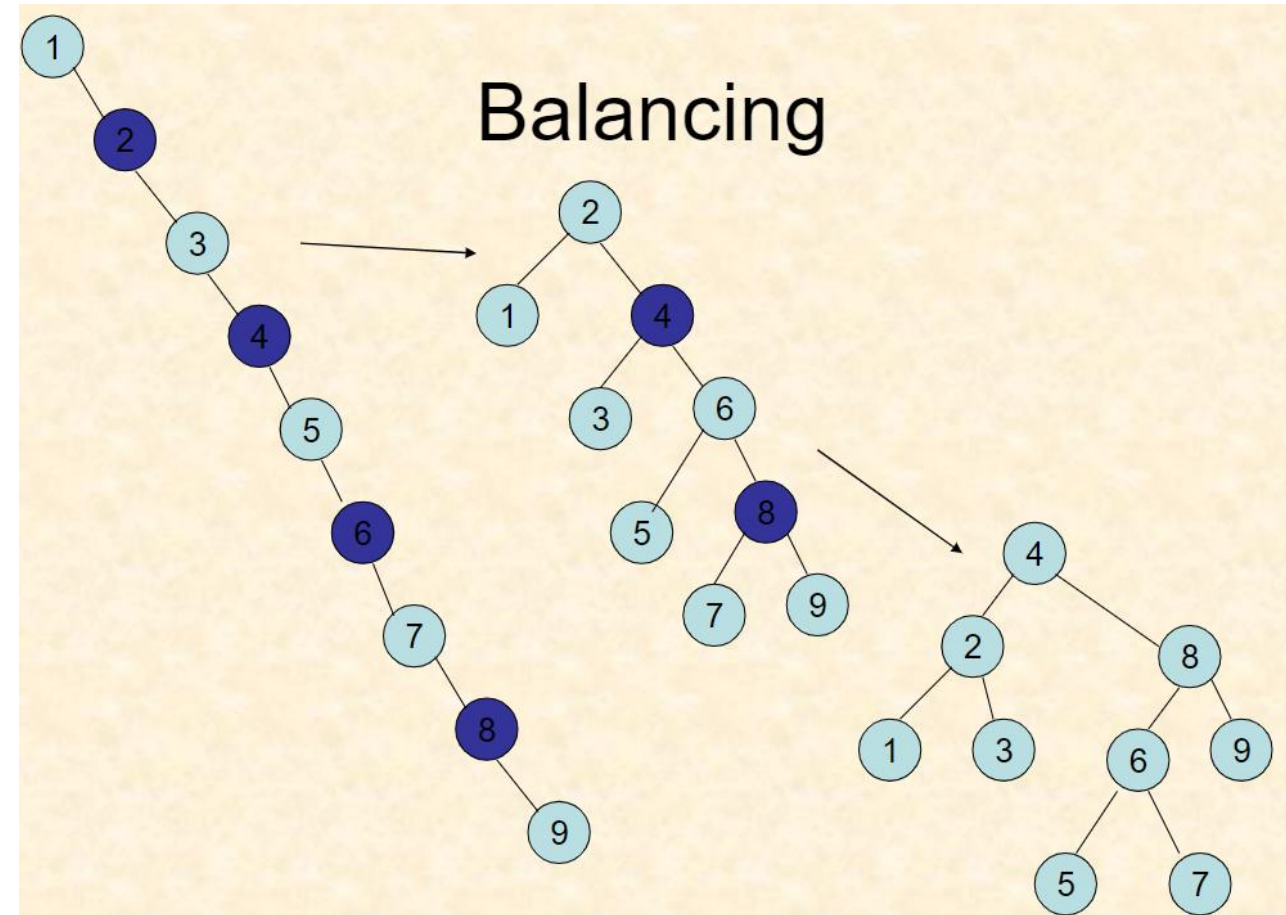
Global tree balancing -The DSW algorithm

- **Phase 1:** Transforming a binary search tree into a backbone



Global tree balancing -The DSW algorithm (cont..)

- Phase 2: convert the backbone or vine (linked list like a tree) into a perfectly balance tree.
- Every other node is rotated around its parent
- This process is repeated down the right balance until a balanced tree is reached.



Global tree balancing -The DSW algorithm (cont..)

- The DSW algorithm is effective at balancing the entire tree (DSW balances the tree in linear time ,i.e., $O(n)$ for n nodes)
- Sometimes, trees need only be balanced periodically, in which case this cost be amortized.
- An alternative approach is ensured that the tree remains balance by incorporating balancing into any insertion/deletion algorithms (e.g. AVL).

Question 1

Consider the following data set:

10, 5, 2, 1, 7, 15, 12, 20

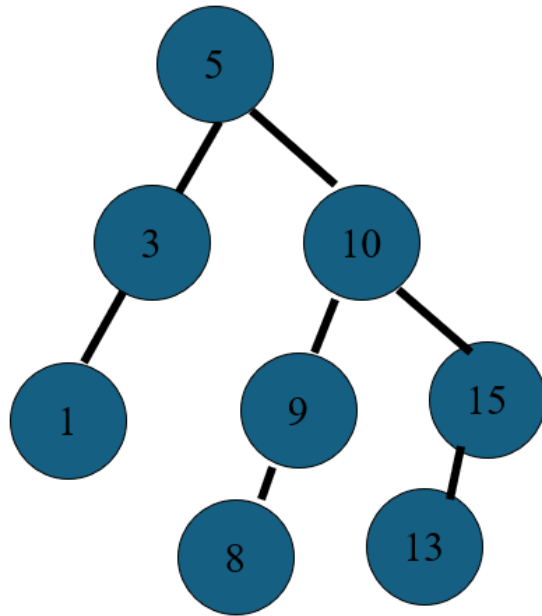
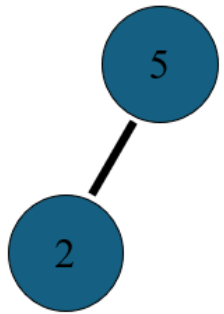
1. Create an Un-balanced Binary Search Tree
2. Is the created tree balanced or not? If not, what is the reason(s) for the imbalance?
3. Transforming a binary search tree into a backbone (vine)
4. **Perform global tree balancing** using the **DSW algorithm (Day-Stout-Warren)** on the vine (backbone) created in above 2.

AVL trees

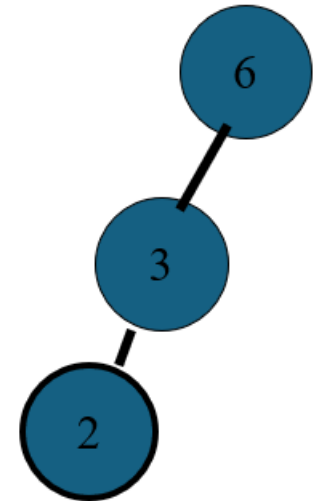
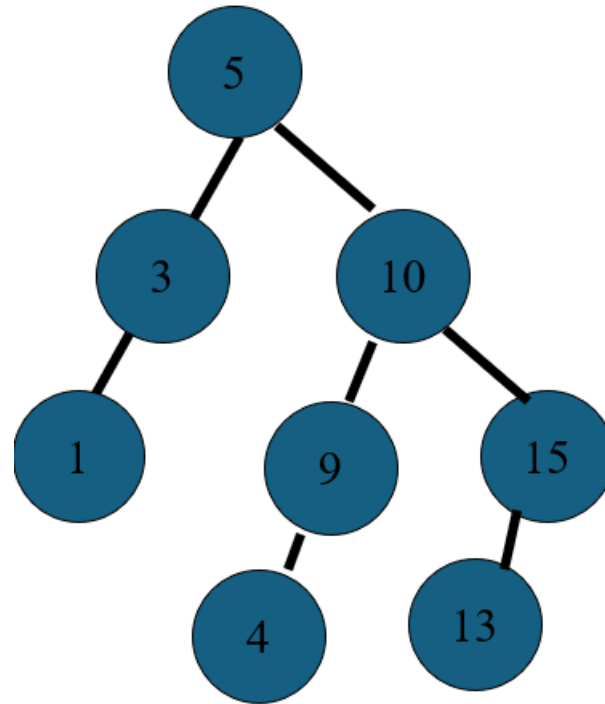
- AVL tree is a self-balancing binary search tree invented by G.M. **A**delson-**V**elsky and E.M. **L**andis in 1962
- Definition :
- An AVL tree is a binary search tree with the additional balance property that, for any node in the tree, the height of the left and right sub-trees can be differ by at most 1.
- *Balance factor = Height (left sub-tree) – Height (right sub-tree)*
- The height of the empty sub-tree is -1
- Balance factor = |Height of the left sub-tree – height of the right sub-tree|
- **-1 ≤ balance factor ≤ 1**

Examples for AVL and Non-AVL trees

- AVL



- Non-AVL

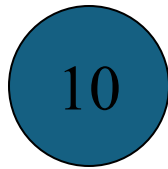


Algorithm : Inserting new nodes into an AVL trees

1. Insert the node in the same way as in an ordinary binary search tree.
2. Beginning with the new node, trace a path back towards the root, checking the difference in height of the sub-trees at each node along the way.
3. If you find a node with an imbalance (a height difference other than 0,+1,-1), stop your trace at this point.
4. Consider the node with the imbalance and two nodes on the layers immediately below this point on the path back to the new node.
5. If these three nodes lie in a straight line, apply a single rotation to correct the imbalance.
6. If these three nodes lie in a dog-leg pattern, apply a double rotation to correct the imbalance.

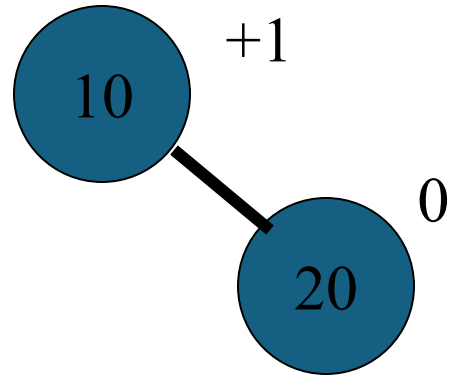
Construction of an AVL trees.

- Eg. 10,20,30,25,27,7,4,23,26,21
- We begin by inserting integer 10 into the root

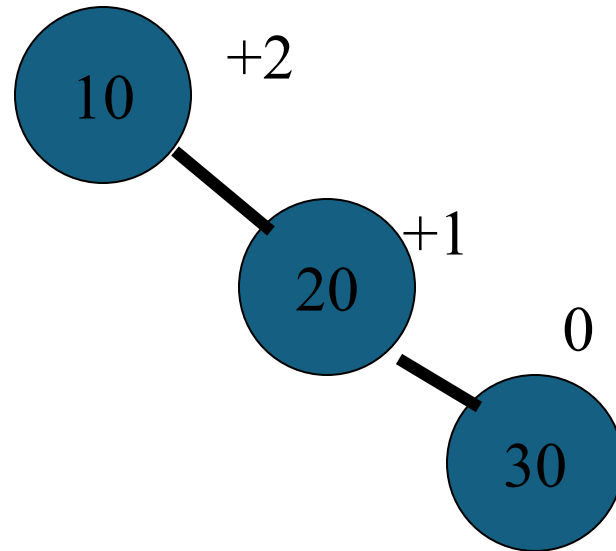


- This node satisfies the AVL condition.

Now we add another node 20



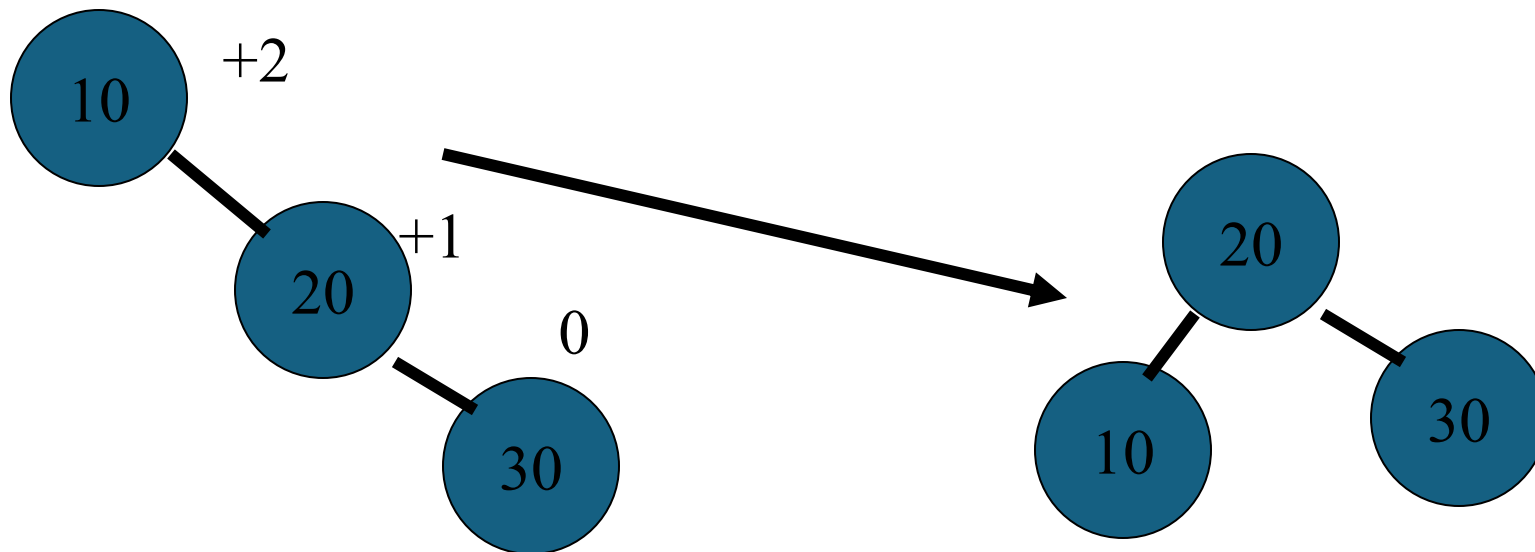
Now we add third node, 30



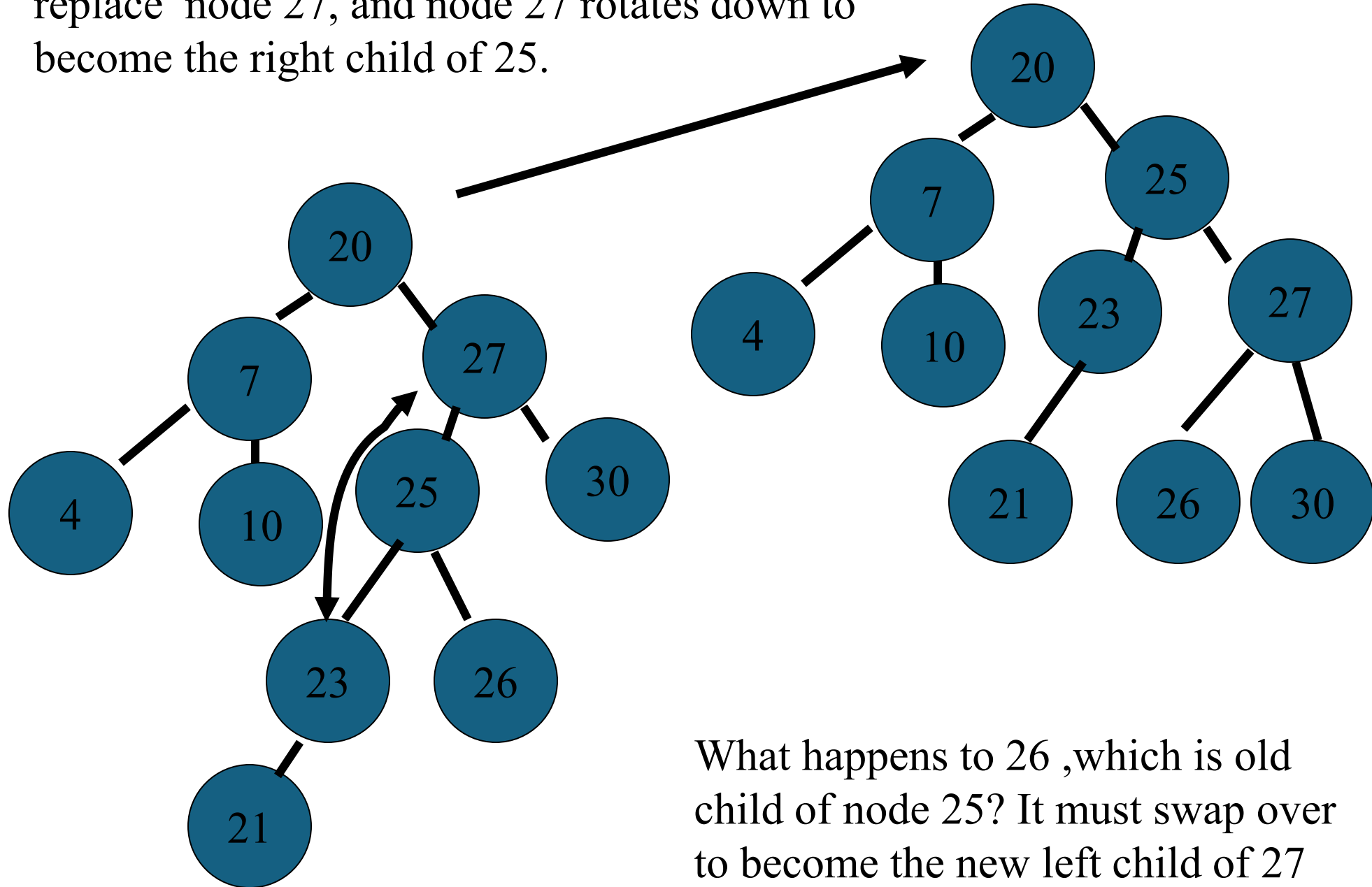
The root node 10 has a difference of 2, which violates the AVL conditions.

Single Rotation :

- This involves shifting the middle node up to replace the top node down to become the left child of the middle.

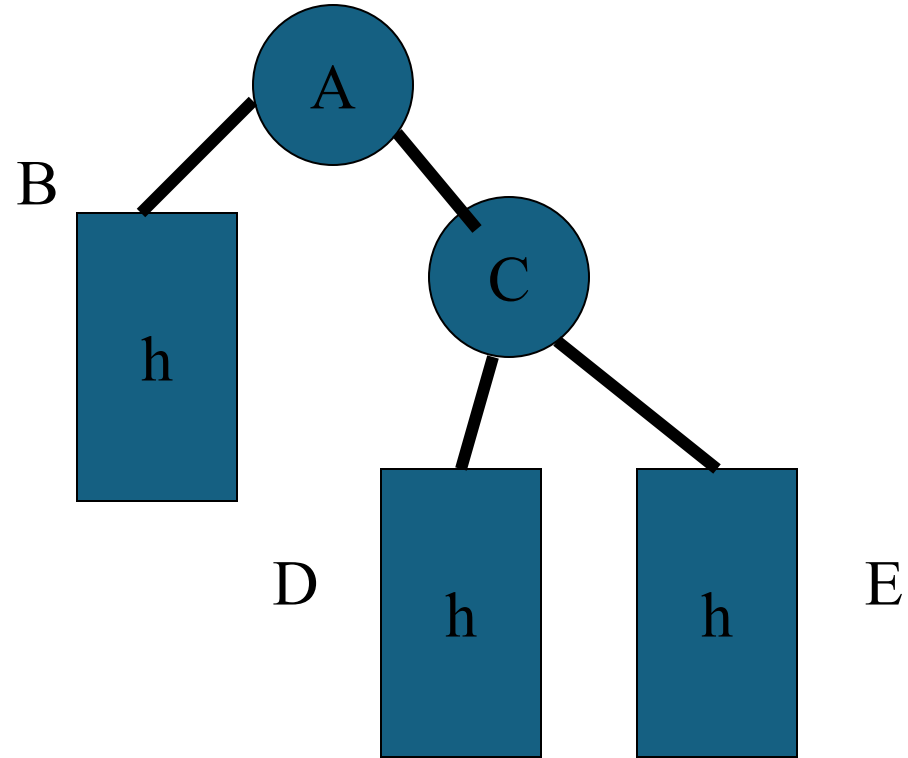


Final Stage : The middle node (25) rotate up to replace node 27, and node 27 rotates down to become the right child of 25.



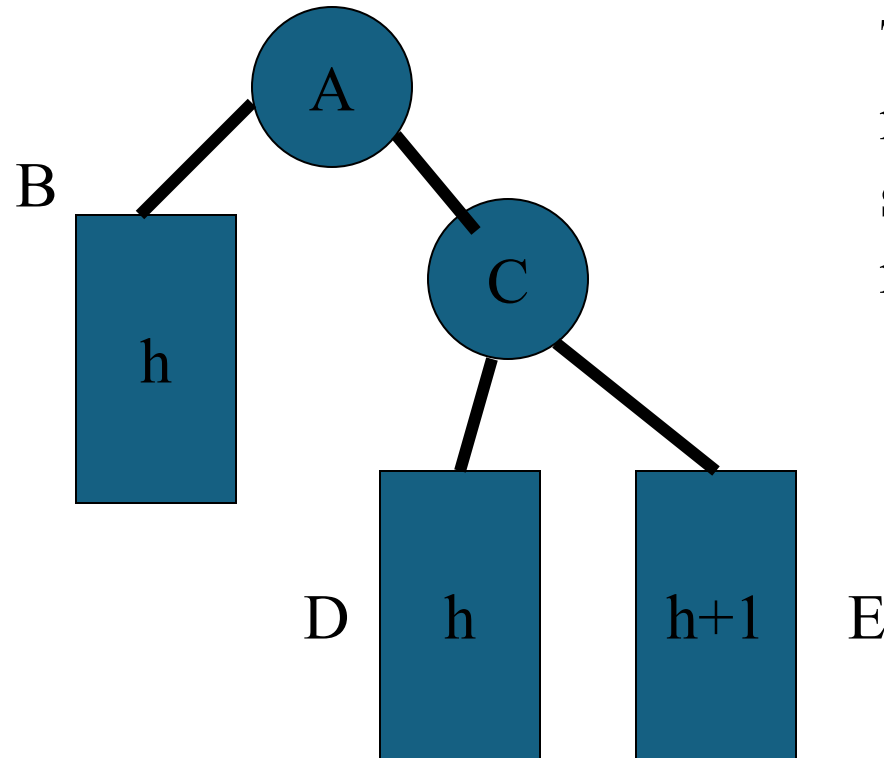
What happens to 26 ,which is old child of node 25? It must swap over to become the new left child of 27

Single Rotation



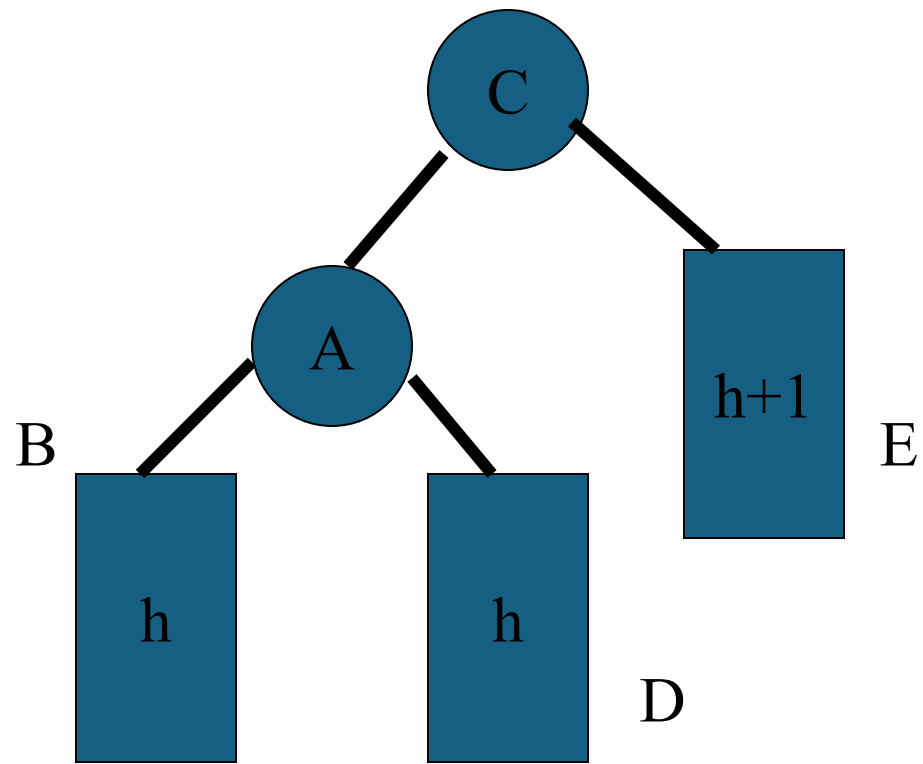
Suppose we want to insert a node into the subtree under node E

The tree is imbalanced at node A



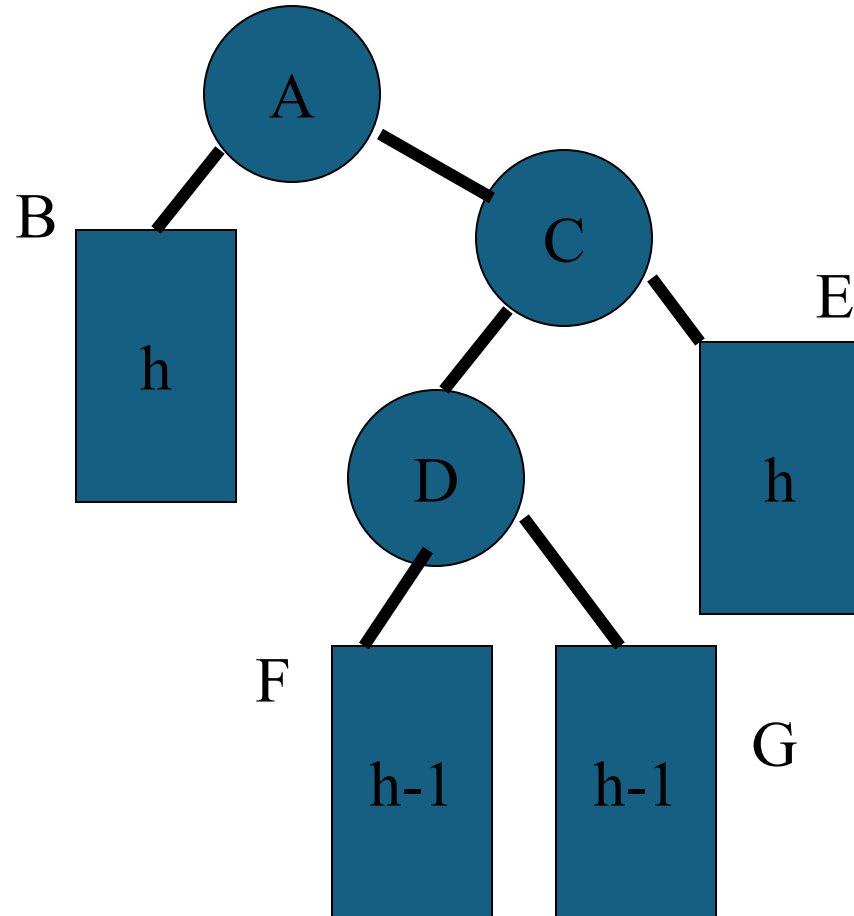
The two nodes immediately below node A. Nodes A, C, E are in the straight line. So we need a single rotation.

We rotate C up to replace A and A down become the left child of C. Node D must swap over to become the new child of A



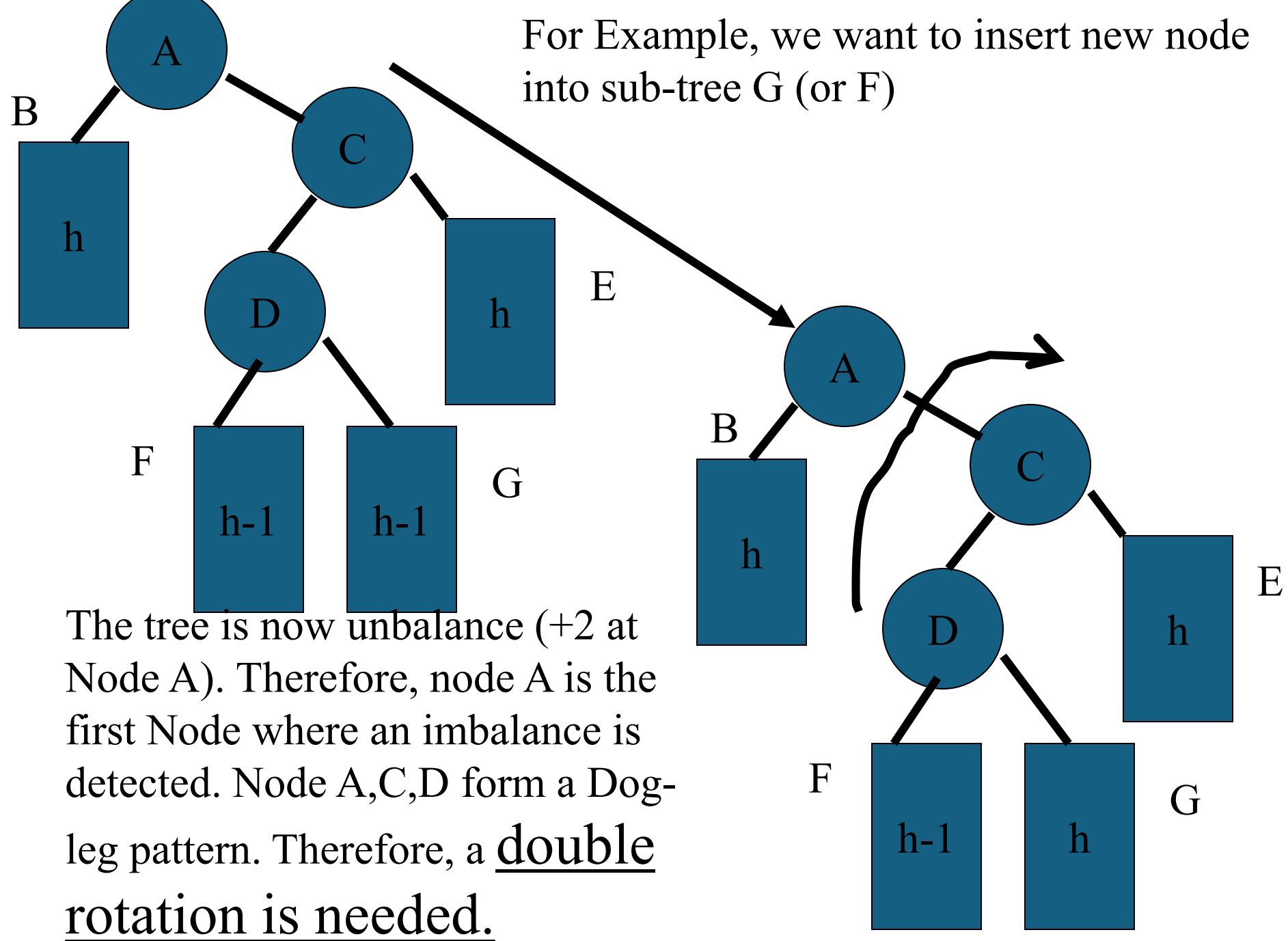
This tree is balanced.

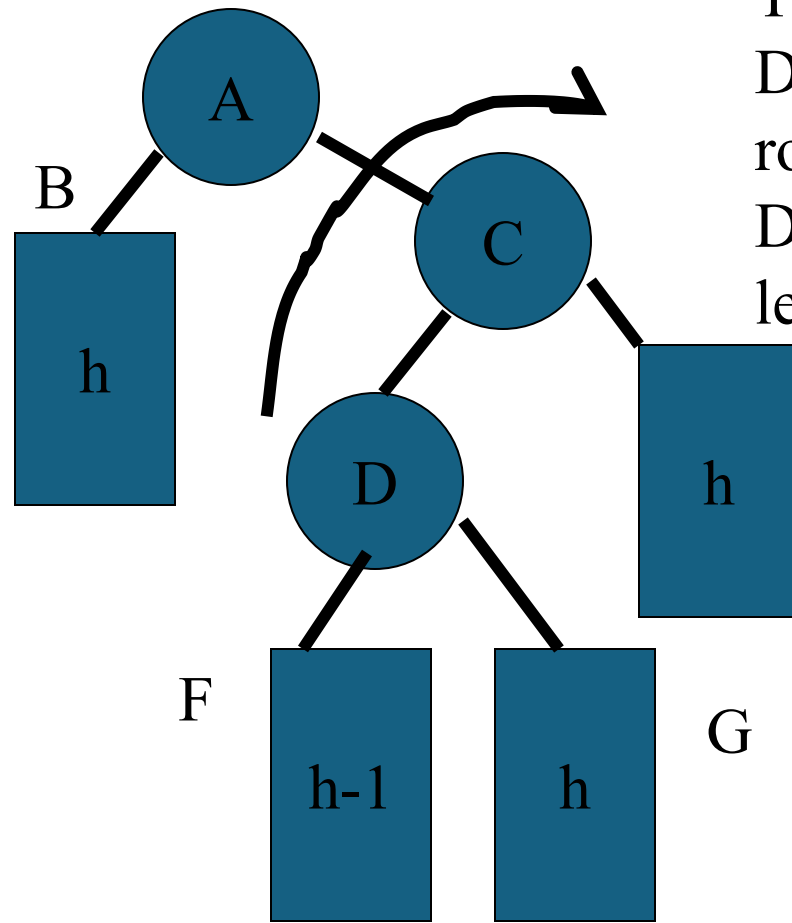
Double Rotation



This tree is an AVL tree,
Assuming all sub-trees in the
rectangular boxes are AVL
trees. The height differences
are 0 at node D & C and +1 at
Node A.

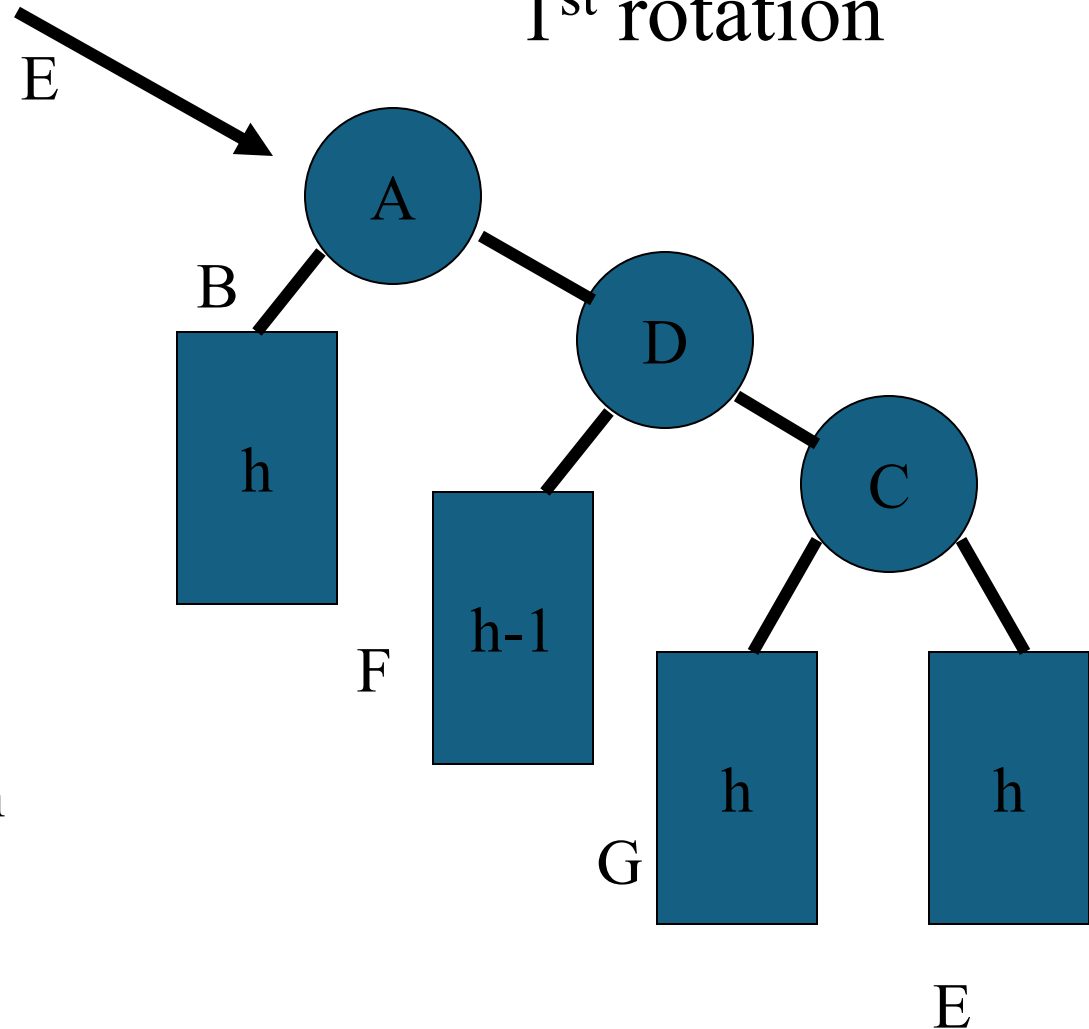
For Example, we want to insert
new node into sub-tree G (or F)



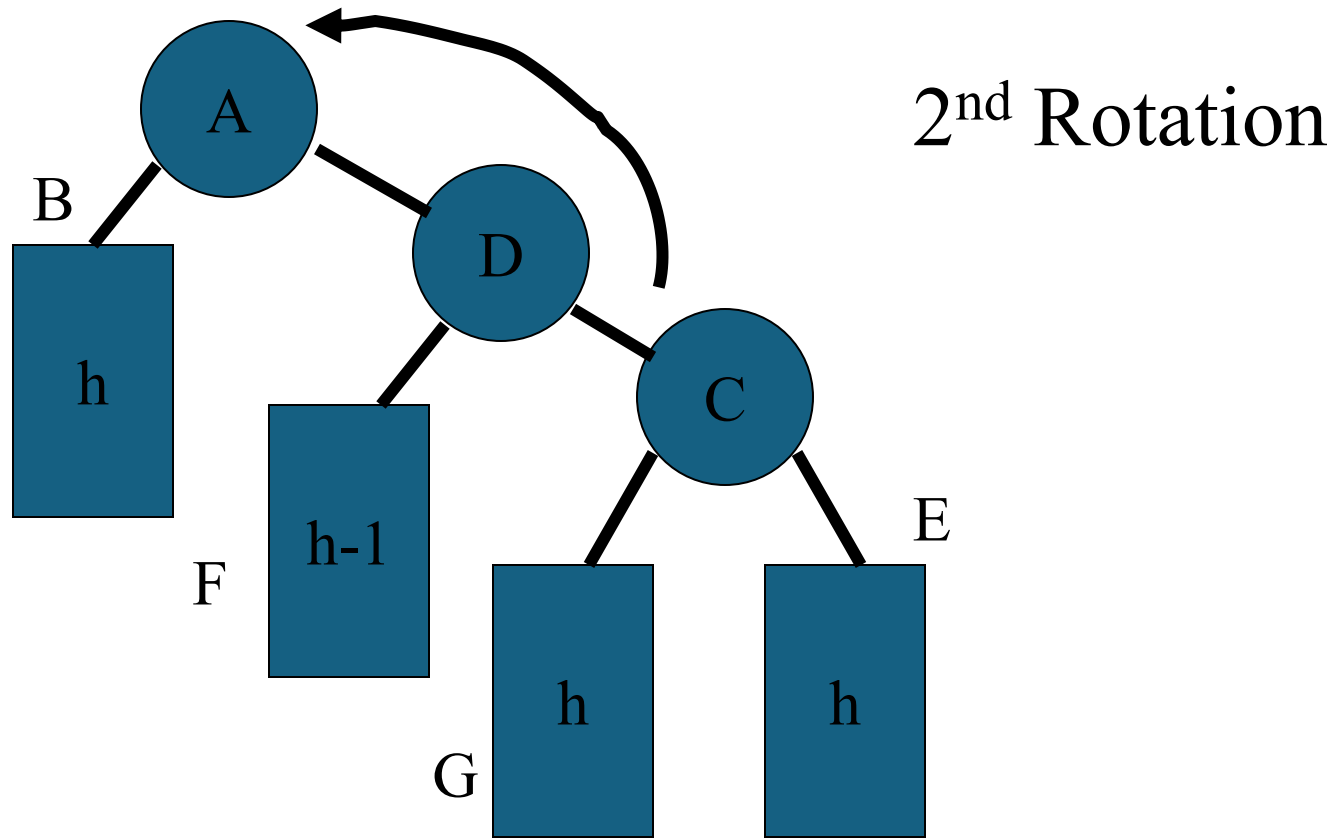


The first rotation involves node C and Node D: Node D rotates up to replace C and C rotates down to become new right child of D. Sub-tree G swaps over to become the left child of node C.

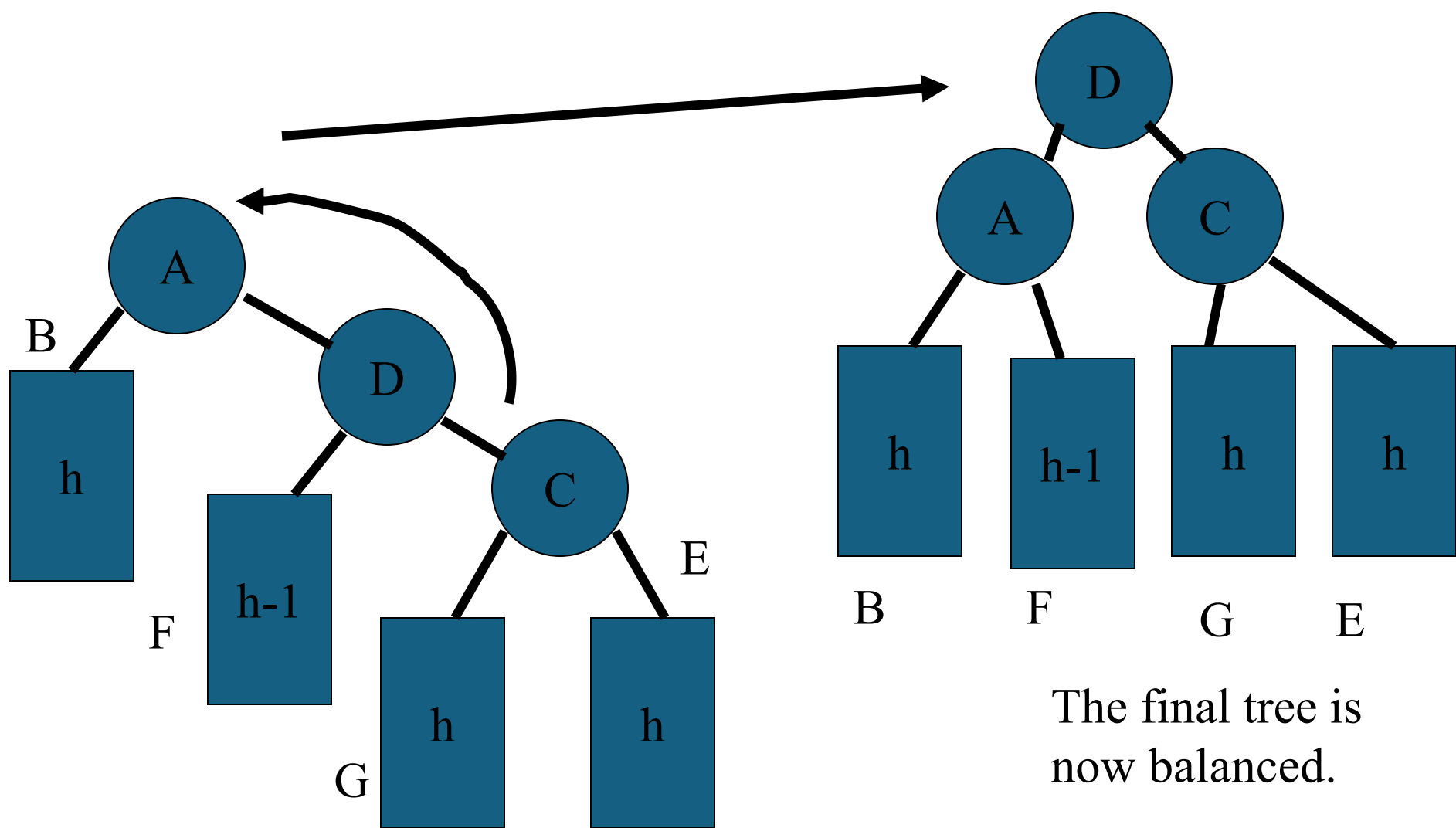
1st rotation



This first rotation does not solve the imbalance problem since the height differences are still +2 at both nodes A and D.



Therefore, we perform the 2nd rotation, which is in the opposite direction to the first one, and which involves nodes A, D and C



Node D rotates up to replace A. Node A rotates down to become the left child of D, and Sub-tree F swaps over to become the new right child of node A.